PROGETTAZIONE DI SOFTWARE SICURO

Lezione 8

Introduzione al testing

Testing basato sui programmi

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Argomenti trattati

Validazione del software

Testing del software

Fondamenti teorici

V&V: Validazione e Verifica

Validazione (convalida): controlla che il sw sia conforme ai requisiti

- accertamento che il software soddisfi i requisiti dell'utente (in termini di funzionalità richieste)
- *building the right system*

Verifica: controlla che l'implementazione corrisponda alla sua specifica

- *building the system right*
- richiede un modello dei requisiti

Validazione a livello di implementazione

Le tecniche di V&V d'implementazione si dividono in

Statiche: collezionano informazioni sul sw senza eseguirlo

- revisioni, walkthroughs, ispezioni, ...
- analisi statica automatica del codice

Dinamiche: analizzano informazioni sul sw eseguendolo

TESTING DEL SOFTWARE

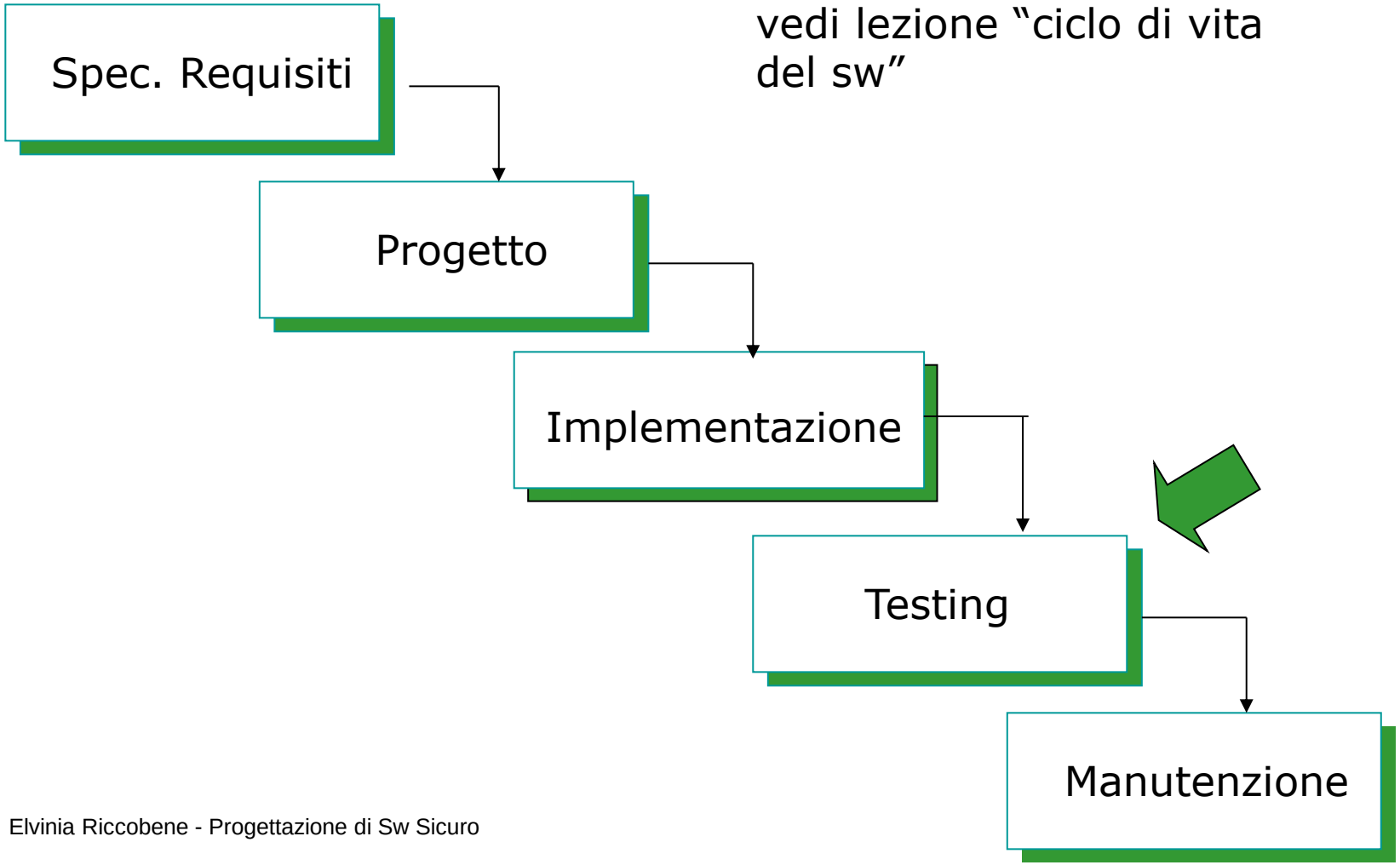
Testing in breve

Il test di un programma o di un sistema consiste (in breve) nell'*eseguirlo* con alcuni casi di test e *controllare* che il comportamento sia corretto

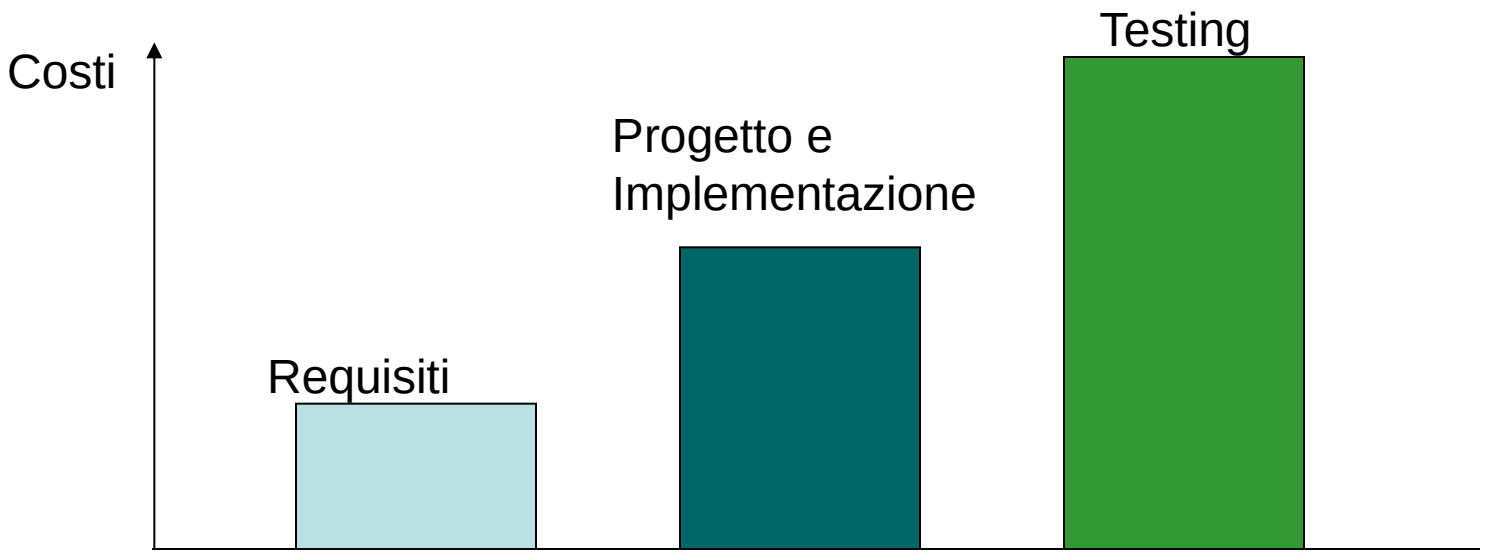
In italiano si dice anche collaudo del software

Ciclo di vita del Software

Ciclo di vita a cascata – vedi lezione "ciclo di vita del sw"



Costi dello sviluppo del Software



I costi del testing sono normalmente molto maggiori che delle altre fasi

Possibili concause sono:

- risparmio sulle fasi iniziali
- frettolosità nel rilascio del software (time to market)

Alcune domande di base

E’ possibile avere software senza difetti?

Tutti i sw devono essere corretti al 100%?

Dipende….

- sistemi critici
- sistemi per la produttività individuale

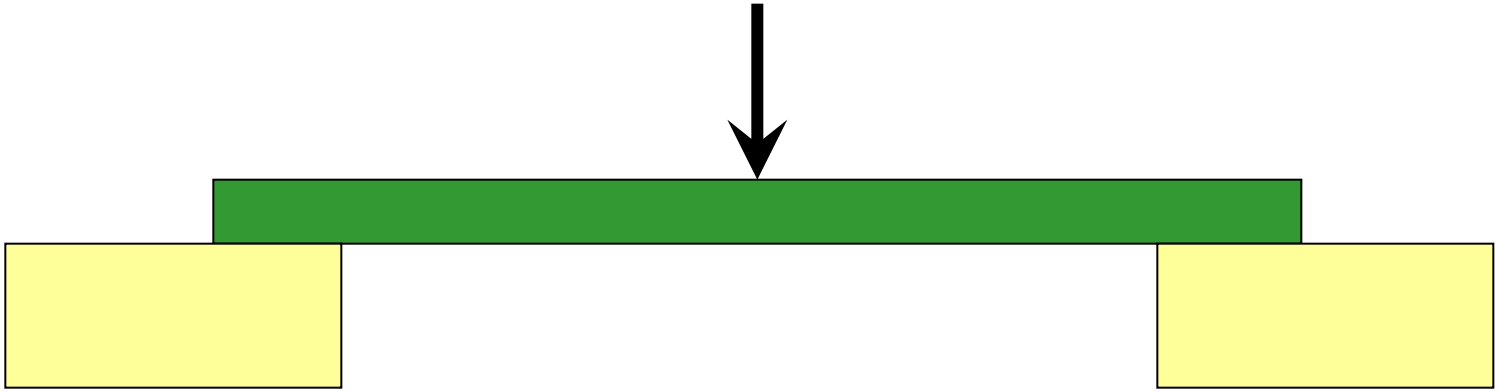
" the software industry : unique industry where selling substandard goods is legal and you can charge extra for fixing the problems."

Test in Ingegneria

Il testing è una pratica diffusa in tutti i campi dell'ingegneria

Però in ingegneria classica gli oggetti hanno un comportamento "continuo":

- un ponte si testa in un punto particolare e se funziona lì funziona tutt'attorno (o addirittura tutto)



Test nell'ingegneria del sw: diversità

1. Il software ha un comportamento molto discontinuo quindi la selezione dei "punti" in cui effettuare il test è molto critica

Esempio

```
a := x / ( x+20)
```

Quale valore prendo di x per vedere se funziona?

2. Un ponte basta che resista, un software basta che non smetta di funzionare o richiedo qualcosa di più? (**robustezza del sw**)

Limiti del testing

Il testing è efficace a trovare difetti ma è inadeguato a provare l'assenza di difetti

- Posso eseguire il programma 1000 volte ed essere (s)fortunato e non scoprire mai un certo bug

Il testing non può sostituire una buona pratica di progettazione e implementazione

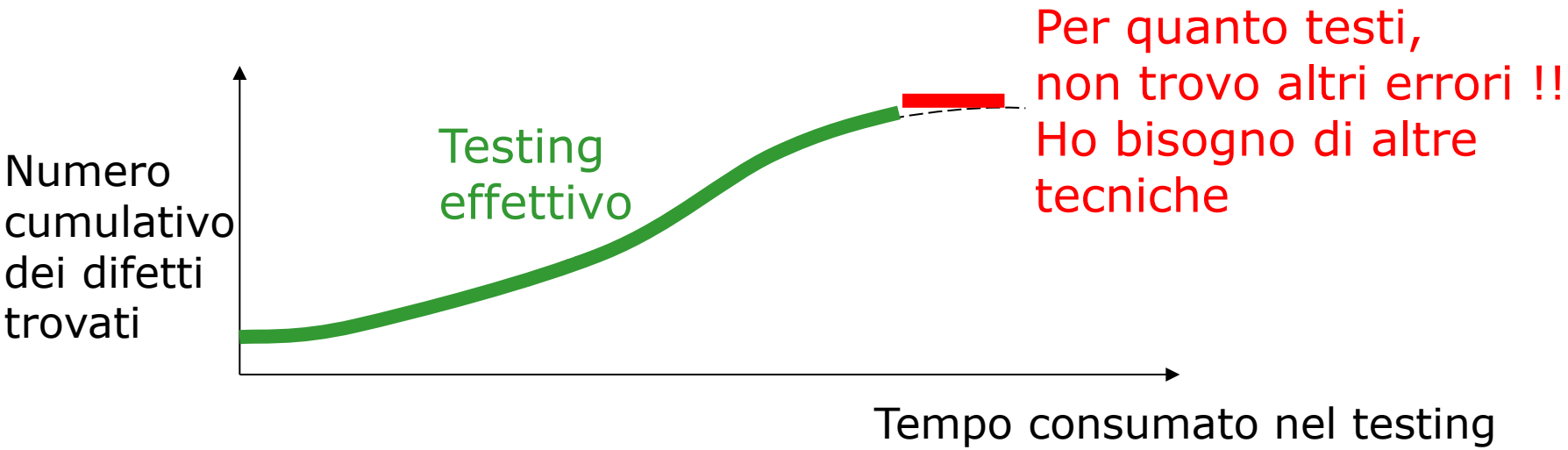
Alcune linee guida generali

Il testing dovrebbe:

- essere automatizzato
  - Uso di tool/sistemi che assistano i programmatori nella *scrittura* di test, nell' *esecuzione*, nella *raccolta* dei dati di output e nell' *analisi* della loro correttezza
- riguardare ogni fase di sviluppo
  - Non solo sul codice finale, ma anche sui requisiti, sui prototipi, …
- essere esteso a tutti i componenti di un sistema
- essere pianificato
  - **test plan** in fase di ing dei reqs
- seguire standard e metodologie dove possibile

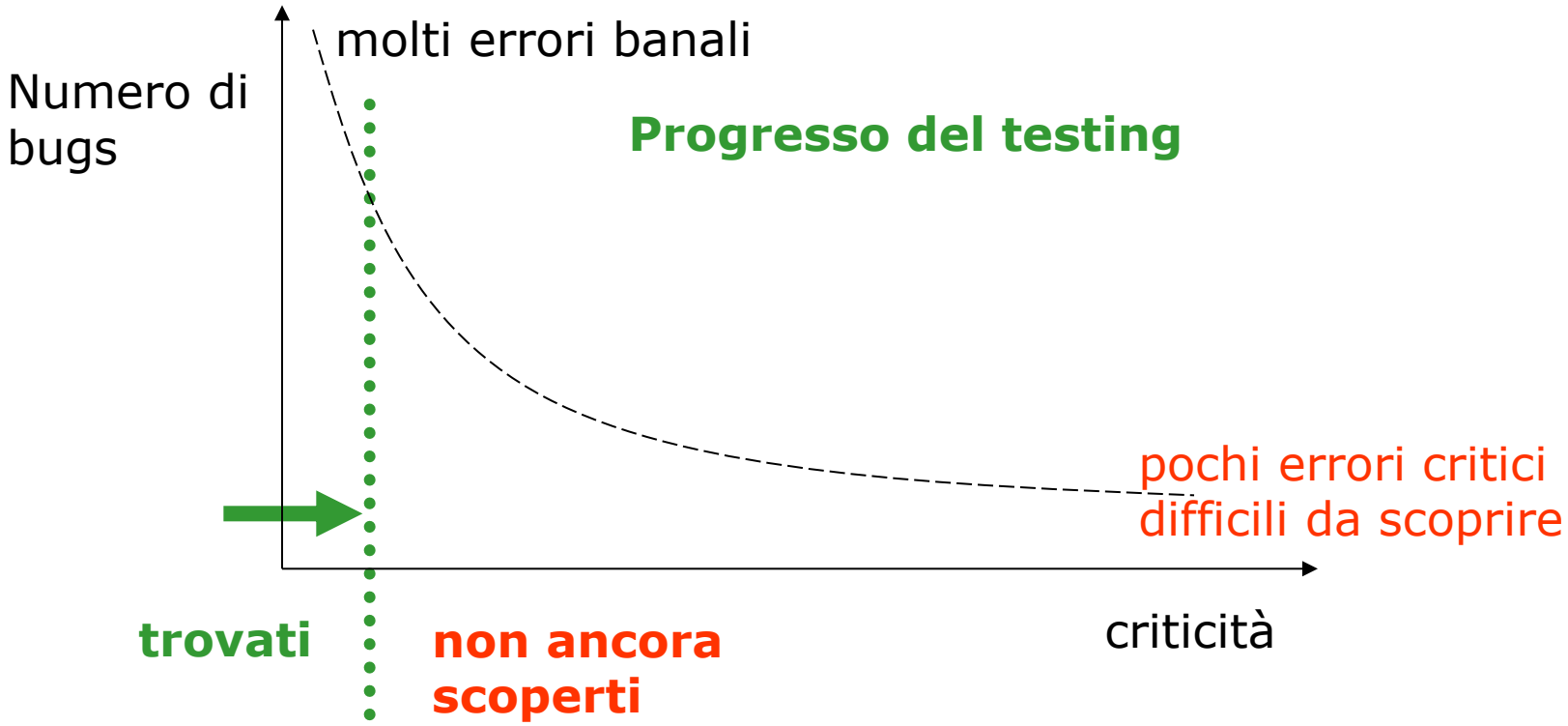
Economia del Testing (1)

Curva di rimozione degli errori



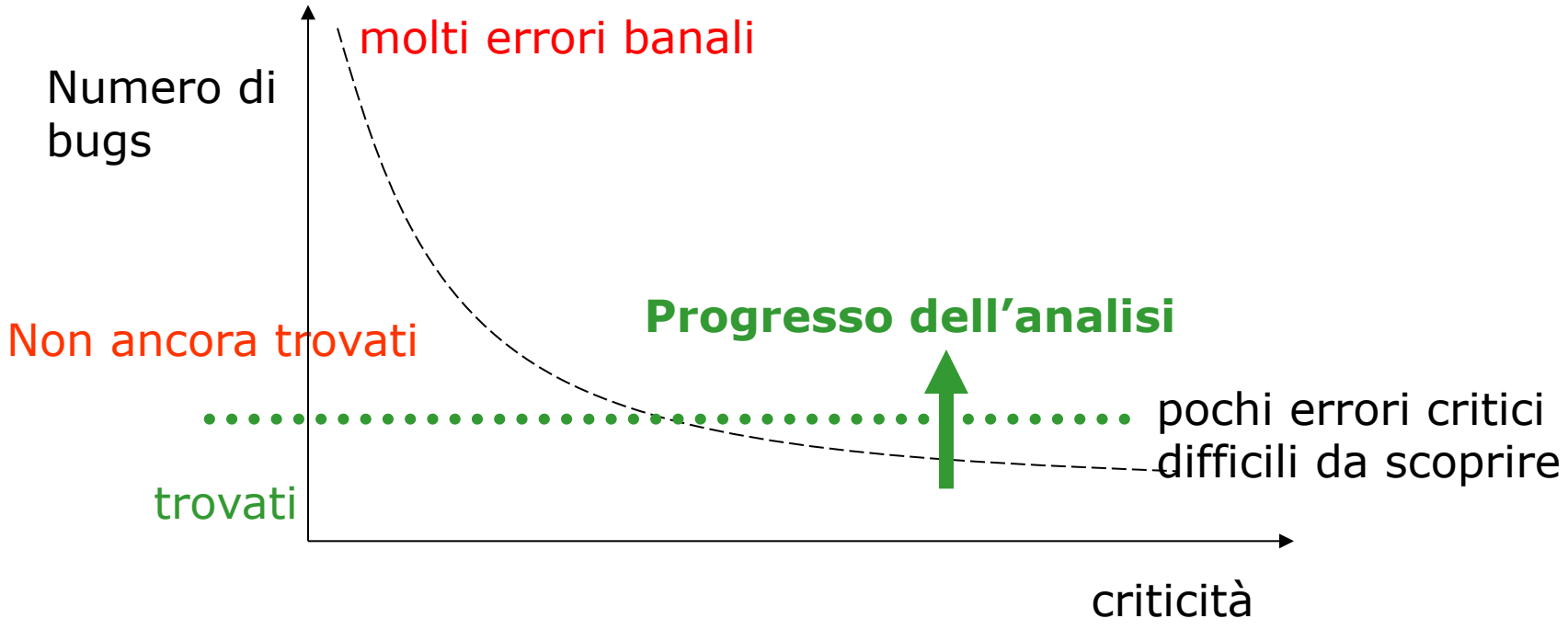
Economia del testing (2)

Il testing tende a scoprire i difetti con probabilità proporzionale alla loro densità**:**



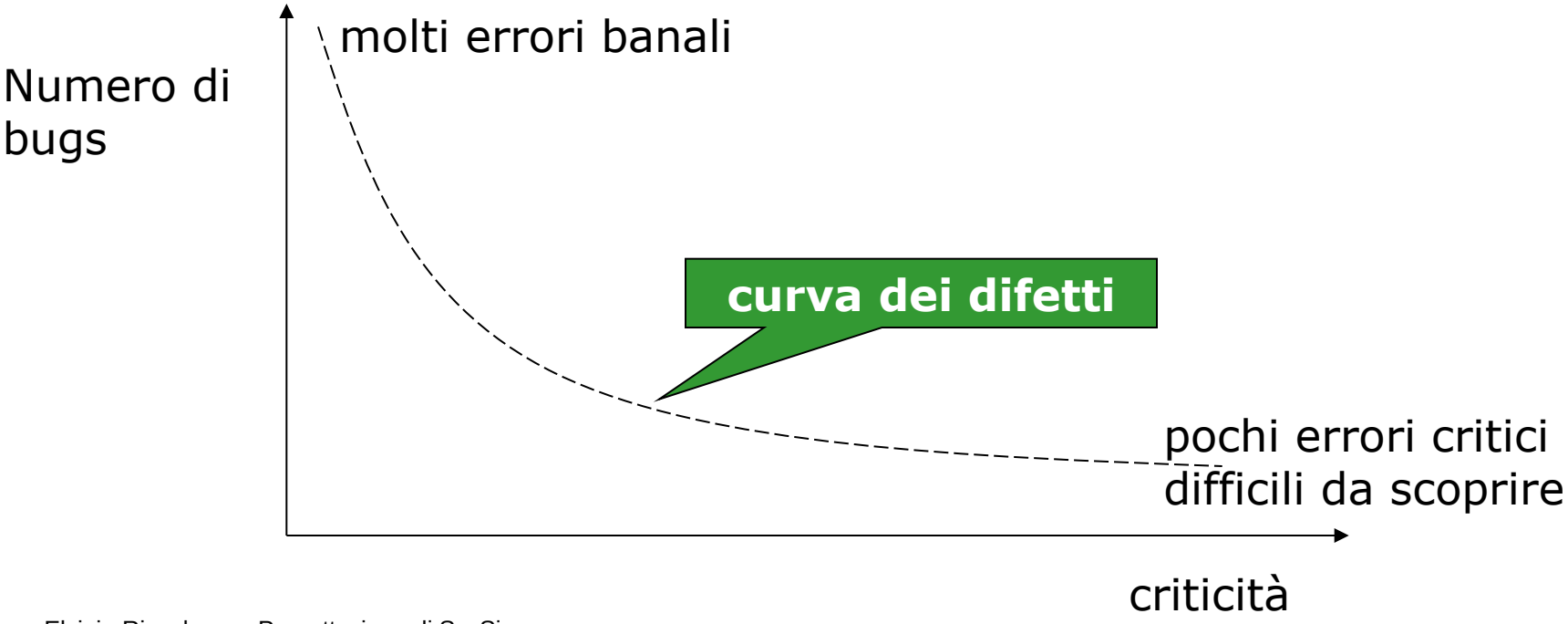
Economia del testing (3)

Tecniche (più costose) di analisi statica cercano di scoprire difetti con probabilità proporzionale alla loro criticità



Curva dei difetti

i difetti più critici sono di meno
e più difficili da scoprire



Economia del testing (4)

Attenti però:

non sempre si ha questa distribuzione (*curva dei difetti*)

- Potrei avere *difetti difficili da scoprire ma non critici*

Fattori da considerare:

1. **quanto costa il testing?**
2. **quanto costa avere un sw con un difetto?**

   = *costo del danno che provoca* x *probabilità che accada*

   - quindi è "inutile" spendere troppo tempo a cercare difetti che non provocano danno
     - Per questo motivo tecniche più costose non vengono impiegate e il sw non critico viene rilasciato con ancora molti difetti

Terminologia base del testing

Dobbiamo concordare alcune parole che useremo da qui in avanti:

- alcuni termini hanno un significato chiaro:
  - testing, debugging, …
- altri sono definiti in modo non ambiguo da standard
  - Ad esempio dalla IEEE Standard Glossary of Software Engineering Terminology
    - difetto, bug, …
- altri sono usati in modo ambiguo
  - Proponiamo un uso e cerchiamo di essere consistenti
    - errore, …

Malfunzionamento

Un **failure** o **guasto** o **malfunzionamento** è il funzionamento non corretto del programma

- legato quindi al comportamento che si osserva durante l'esecuzione

Esempio:

```
// programma che dovrebbe restituire il doppio
// del valore passato come parametro
int raddoppia(int x) {
      return x*x ;
}
```

se chiamo raddoppia(3) noto un malfunzionamento
se chiamo raddoppia(2) non vi e' malfunzionamento

Difetto

Un **difetto** o **anomalia** o **fault** o **bug** è un elemento del programma sorgente non corrispondente alle aspettative

- riguarda quindi la parte statica
- **uno o più difetti possono causare malfunzionamenti**
- Nell'esempio:

```
int raddoppia(int x) {
      return x*x ;
}
```

il difetto e' *x invece che *2.

Nota

- un programma può avere molti difetti e non presentare alcun malfunzionamento.
- scopo del testing e' quello di evidenziare difetti mediante malfunzionamenti.

Errore

Errore: fattore (umano) che causa una deviazione tra il software prodotto e il programma ideale (uno o più errori possono produrre uno o più difetti nel codice)

- **Esempio**: errore di analisi dei requisiti, progetto, battitura,...
- Quando il programmatore commette un errore, il programma ha un difetto che può generare un malfunzionamento

Testing e debugging

Testing di programmi:

- eseguire il programma con dei casi di test e analizzare i risultati per trovare i difetti (bug)

Debugging:

- correggere i difetti ed eventualmente scoprire gli errori

Scopi del testing

Testing di programmi ha due scopi principali:

- valutare l'affidabilità di un sw (*reliability*) e fornire confidenza allo sviluppatore ed al cliente che il sw soddisfa i suoi requisiti (test di accettazione)
  - Almeno un test per ogni requisito specificato
  - Test per tutte le funzionalità per garantire la conformità ai reqs e l'assenza di malfunzionamenti
  - Test dei comportamenti più critici o più frequenti per controllare che causino o meno malfunzionamenti
- mettere in evidenza i difetti (*bug*) mediante i malfunzionamenti, per scoprire eventuali errori (test dei difetti):
  - cercare quei comportamenti che mettono in evidenza difetti

Test di accettazione e test dei difetti

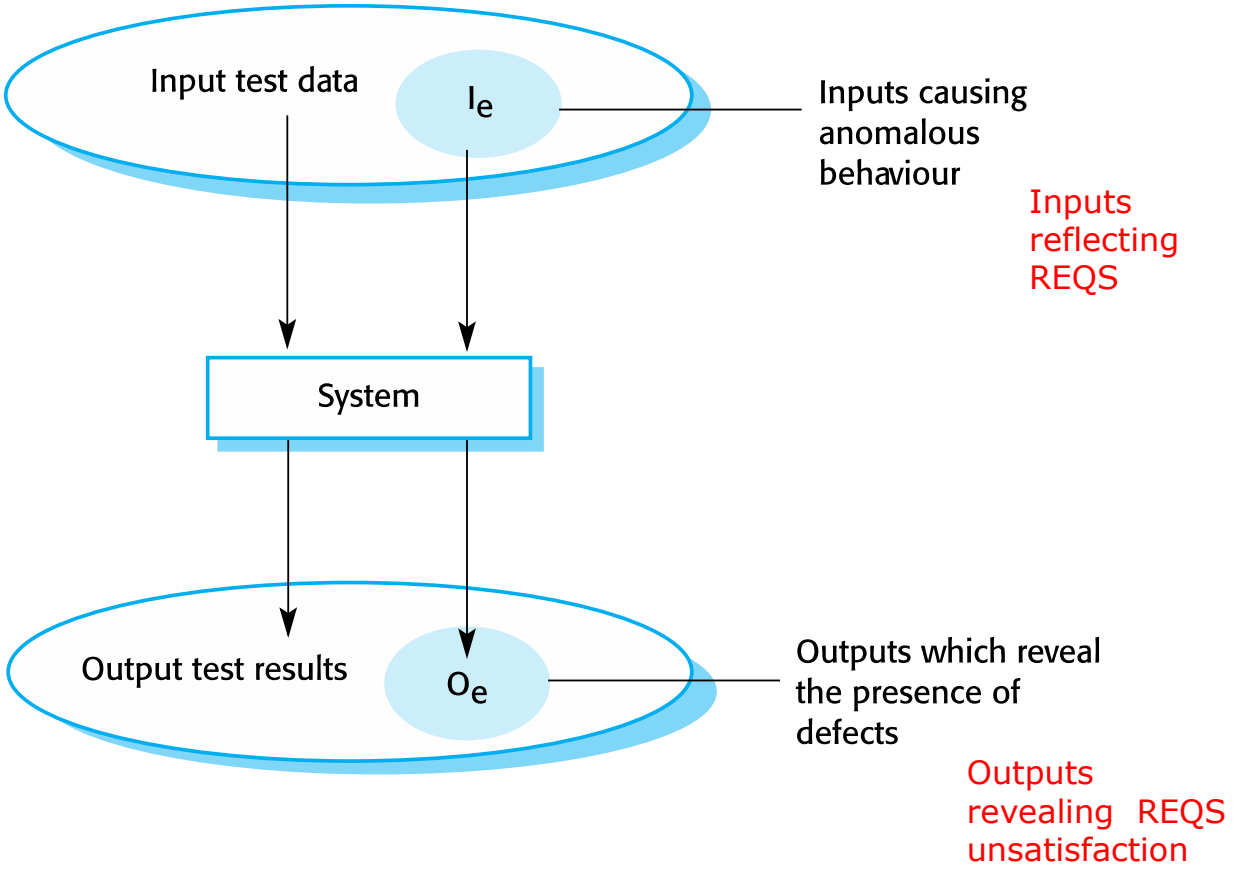
Il primo obiettivo porta al test di accettazione

- Si definisce un set di test case che riflettono l'uso previsto del sistema.
- La mancanza di malfunzionamenti garantisce che il sistema funzioni correttamente.

Il secondo obiettivo porta al test dei difetti

- I test case sono progettati per esporre i difetti.
- I casi di test nel test dei difetti possono essere deliberatamente oscuri e non devono necessariamente riflettere il modo in cui il sistema viene normalmente utilizzato.

Modello input-output di test di programma



- Il test dei difetti ha interesse negli input che causano output errati
- Il test di accettazione ha interesse negli input che causano output non attesi dai reqs

Tre stadi di test

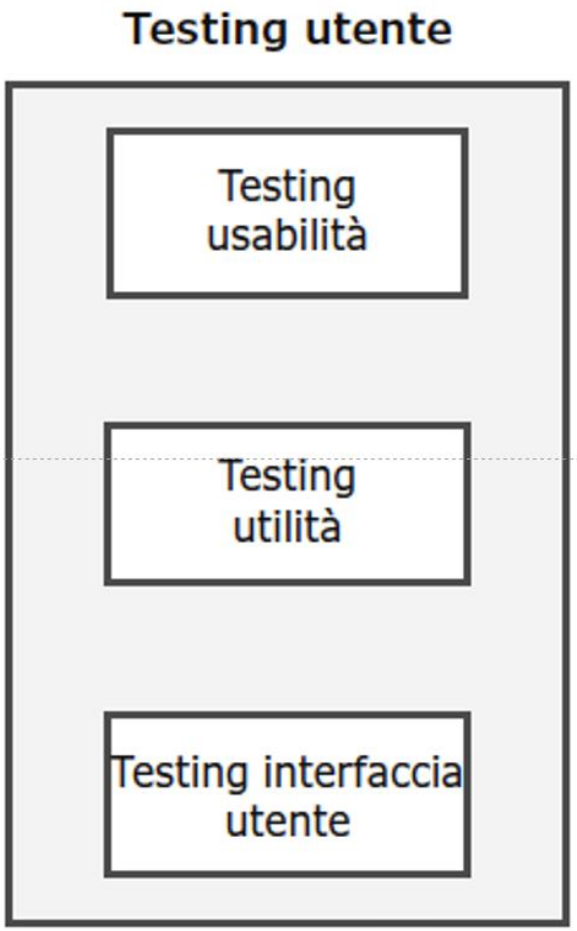
Test di sviluppo (o funzionale), in cui il sistema viene testato durante lo sviluppo per scoprire bug e difetti.

Test di rilascio (o release), in cui un team di test separato testa una versione completa del sistema prima che venga rilasciata agli utenti.

Test degli utenti, in cui gli utenti o potenziali utenti di un sistema testano il sistema nel proprio ambiente.

- Tra questi il test di accettazione e quello di usabilità

Testing utente



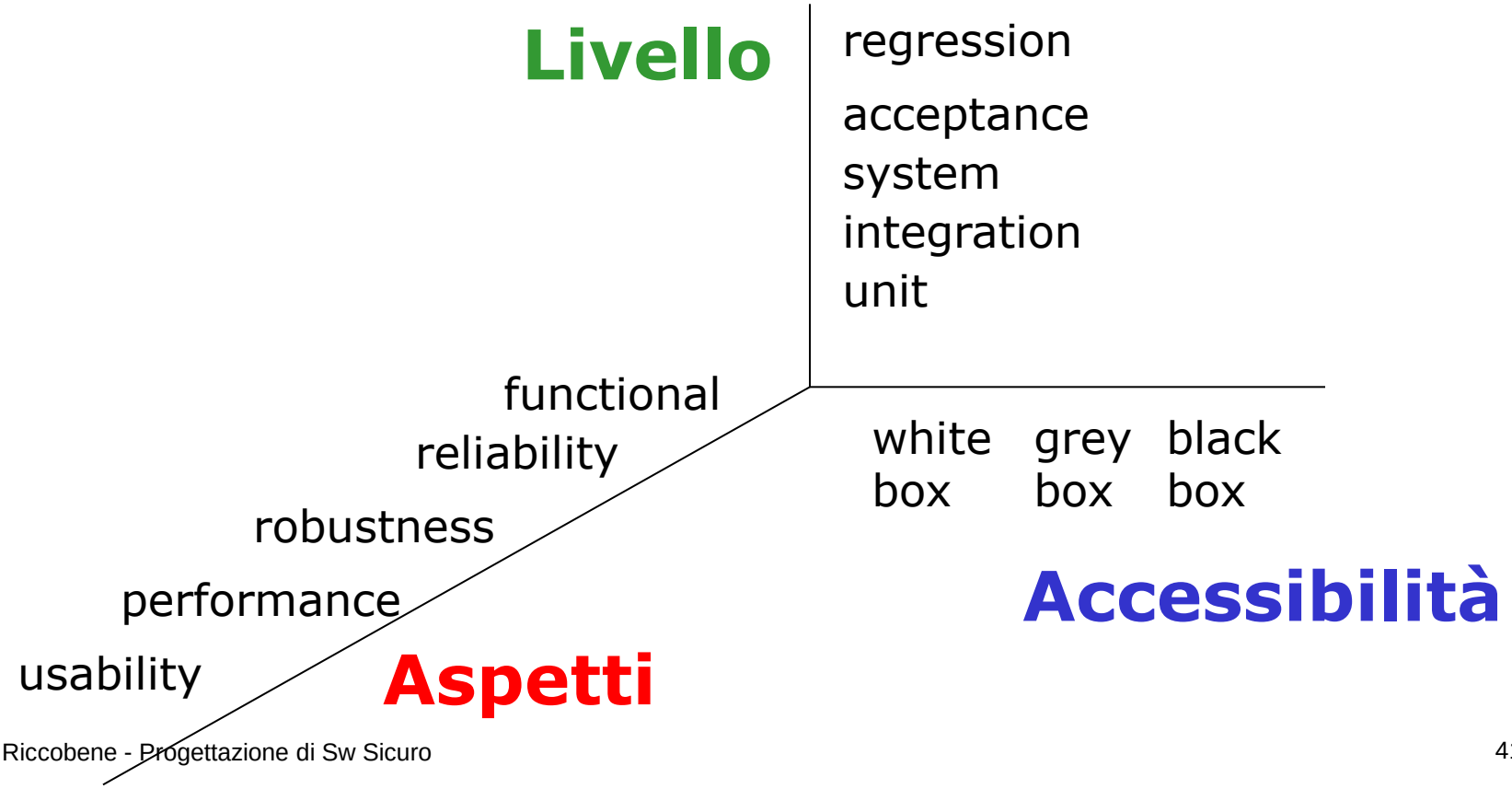
Verifica che gli utenti riescano a imparare velocemente a usare il sistema.
Verifica che gli utenti riescano a usare il sistema senza commettere errori.

Verifica che le feature di sistema permettano agli utenti di fare quello che desiderano.
Verifica che l'insieme di feature sia adatto alle attività che l'utente deve svolgere.

Verifica che gli utenti gradiscano il look and feel dell'interfaccia utente del sistema.

Tipi di testing

Esistono tanti tipi di testing a seconda del livello a cui si effettuano, del tipo di accesso al sistema da testare e degli aspetti che si vogliono testare



I livelli del Testing di sviluppo (livello)

livello da testare	Tipo di test
Ciò di cui l'utente ha bisogno (reqs)	Acceptance testing
Specifica	Conformance testing
Progetto	Integration testing
Codice	Unit testing

Livelli di granularità

Test di

accettazione: il comportamento del software è confrontato con i requisiti dell'utente finale

conformità: il comportamento del software (tutto) è confrontato con le specifiche dei requisiti

sistema: controlla il comportamento dell'intero sistema (hw + sw) come monolitico

integrazione: controllo sul modo di cooperazione delle componenti (come previsto dal progetto); si concentra sul test delle interfacce dei componenti

unità: test del comportamento delle singole unità di programma o classi di oggetti. Il test unitario dovrebbe concentrarsi sul test della funzionalità di oggetti o metodi

regressione: test del comportamento di release successive

Unit Testing

Testa il codice a livello di singola unità

In OO (come JAVA)

- la singola unità è una classe
- UT testa
  - I singoli metodi delle classi
  - Metodi di set e get di tutti gli attributi degli oggetti
  - L'oggetto in tutti gli stati possibili e le sue transizioni

Per ogni metodo testato - detto test unit - si introducono

- test driver: metodo che chiama il test unit con opportuni parametri
- test stub: (opzionale) metodo che sostituisce eventuali metodi usati dal test unit per testare in modo isolato e controllato
  - La scrittura di stub può essere onerosa e viene spesso evitata

Unit Testing - Esempio

Esempio

```
foo(int x2, int y2) {
……
gig(x2+2);
……
}
```

→ **`foo`: test unit**

Metodo da testare

```
testFoo() {
……
foo(x1+1, y1-1);
// controllo
}
```

→ **`testFoo`: test driver**

Metodo che testa foo

Simula una unità chiamante

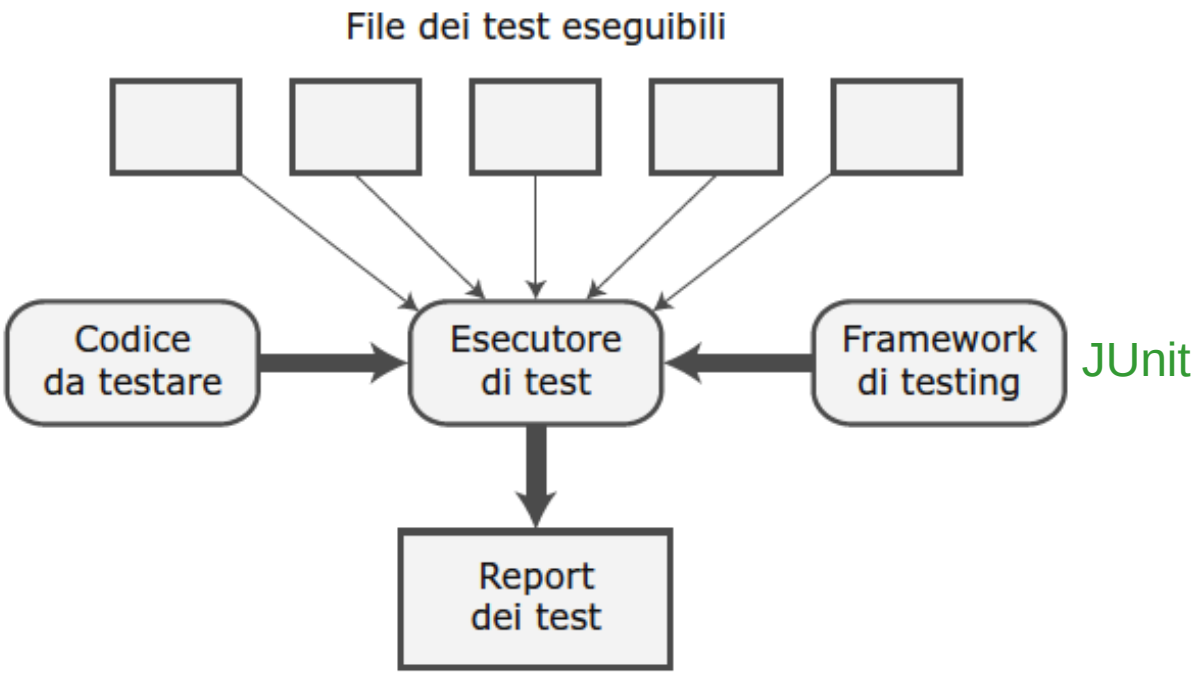
```
gig(int x3) {
      ……
}
```

→**`gig`: test stub** (opzionale)

Simula un metodo chiamato da foo in modo di isolare il caso di test dal resto del sistema

Unit Test automatizzato

- Si utilizza un **framework di automazione dei test** (come JUnit) per scrivere ed eseguire i test del programma.



Unit Test automatizzato

- Il framework di unit test fornisce classi di test generiche (es. *TestCase*) da estendere per creare casi di test specifici. Consentono:
  - **Una parte di configurazione** (*setup*), in cui si inizializza il sistema con il test case, ovvero gli input e gli output previsti.
  - **Una parte di chiamata** (*azione*), in cui si chiama l'oggetto o il metodo da testare.
  - **Una parte di *asserzione*** in cui si confronta il risultato della chiamata con il risultato atteso.
    - Se l'asserzione restituisce true, il test ha avuto esito positivo se false, quindi ha fallito.

Test d'integrazione (1)

Testa l'integrazione tra le diverse unità/componenti

Esempio:

- compatibilità dei tipi importati/esportati
  - spesso eseguita dal linker o dal compilatore
- compatibilità nei domini di import/export

  Esempio:

  - F1 chiama F2 con un parametro intero
  - F2 assume che il parametro sia >0, mentre F1 chiama con parametro = 0
- compatibilità di dati importati/esportati

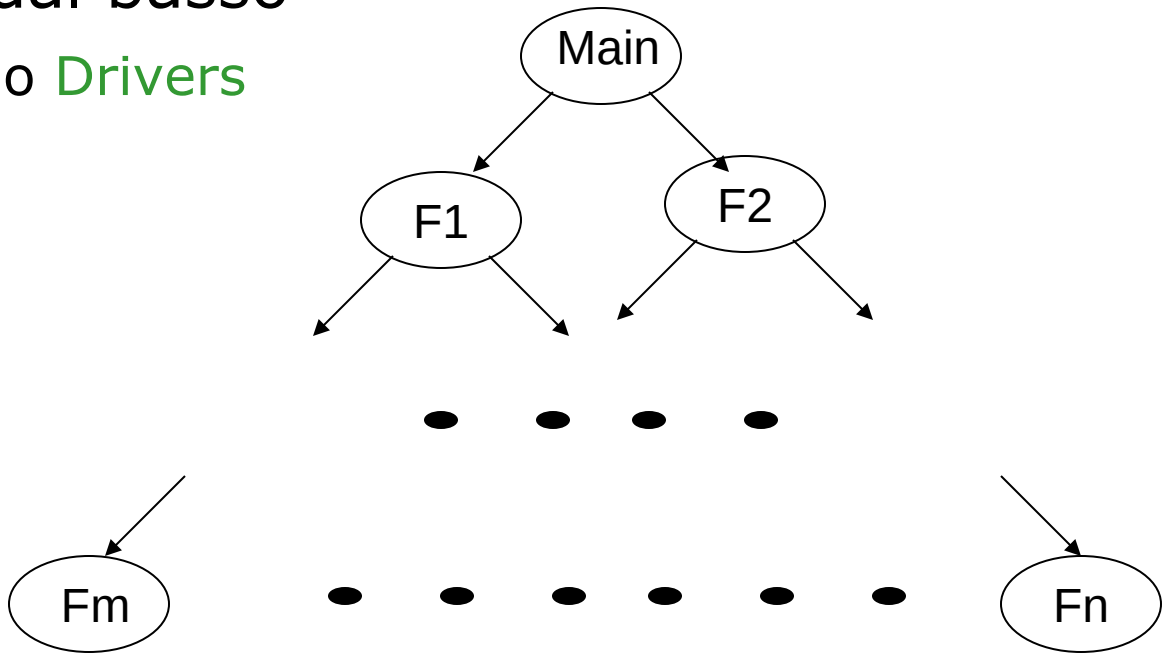
  Esempio:

  - F1 chiama F2 e con un parametro Elapsed_time
  - F1 pensa siano secondi, F2 invece millisecondi
  - Successo veramente su sonda Pioneer!!!
- Spesso può richiedere l'HW

Test d'integrazione (2)

Alcune strategia per il test di integrazione

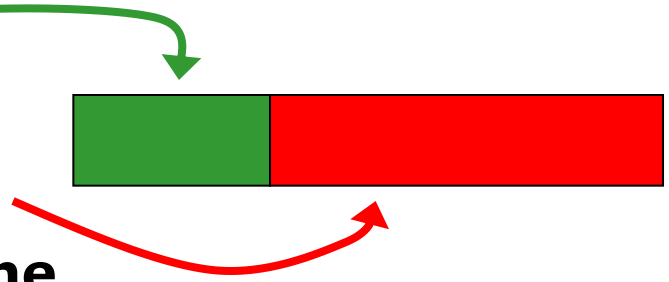
- top-down: inizia dall'alto
  - Si devono sviluppare Stubs
- bottom-up: dal basso
  - Si sviluppano Drivers
- big-bang



Regression Testing

Sviluppo vs Manutenzione

- costi di sviluppo: 1/3
- costi di manutenzione: 2/3

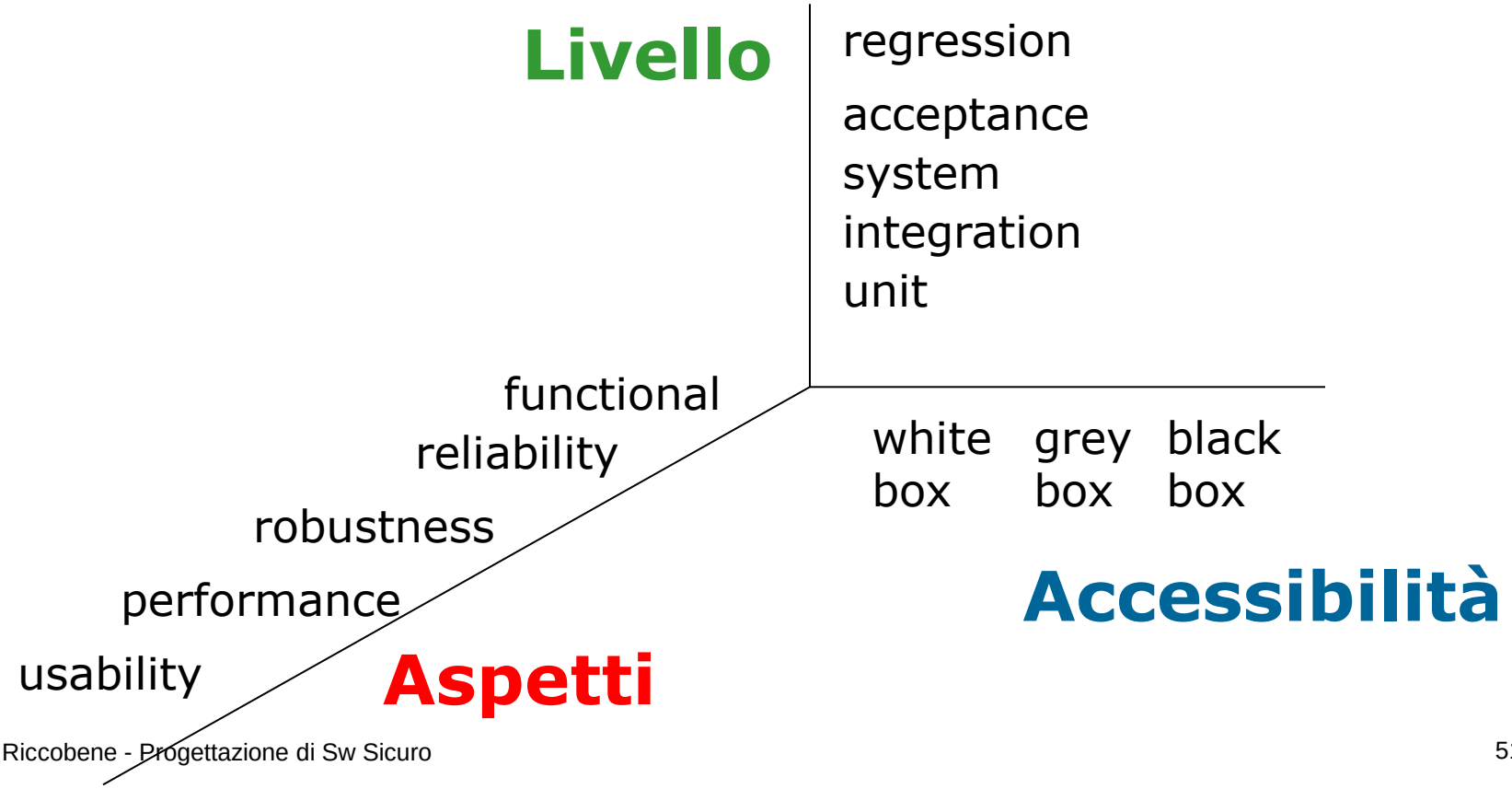


Testing durante la manutenzione

- obiettivi nel testare programmi modificati:
  - accertarsi di aver eliminato i difetti segnalati
  - essere sicuri di non introdurre nuovi difetti
  - evitare di dover buttar via i vecchi casi di test e riscriverne nuovi da capo
  - riusare i vecchi casi di test e confrontare che il comportamento non sia cambiato (eccetto malfunzionamenti)
  - evitare di dover ritestare tutti i programmi anche quelli non modificati

Tipi di testing

Esistono tanti tipi di testing a seconda del livello a cui si effettuano, del tipo di accesso al sistema da testare e degli aspetti che si vogliono testare



Aspetti da testare

Funzionalità

Affidabilità – reliability

Robustezza

Performance

Usabilità

+ Proprietà di sw sicuro

Accessibilità del Testing

A seconda del tipo di "accesso" all'unità testata si ha:

white box testing, o structural testing, o program-based testing

- si assume che il programma sorgente sia disponibile

black box testing, o functional testing, o specification-based testing

- si assume che non si guardi il programma sorgente ma solo quello che dovrebbe fare

grey box testing

- un mix tra i due

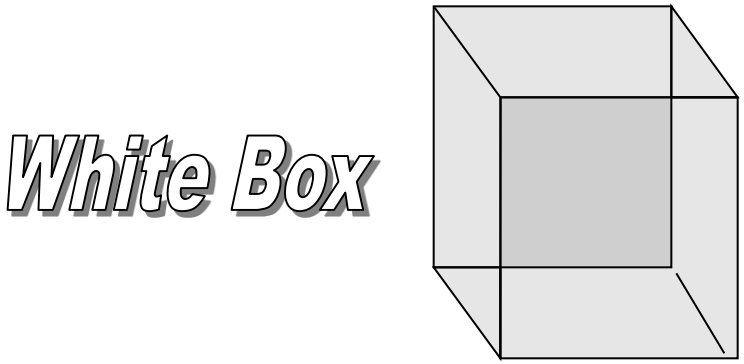
White box testing

E' basato sulla struttura interna del programma

- deriva i casi di test dal programma
- controlla e osserva i programmi durante l'esecuzione

In genere analizza "quanto codice è stato eseguito" o coperto

- non garantisce che il programma faccia quello effettivamente richiesto
- ci sono diverse alternative che vedremo



Black-Box testing

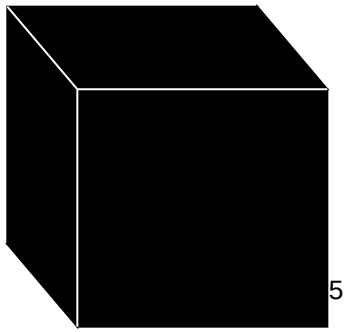
Ignora la struttura del programma e considera solamente i suoi requisiti

- deriva i casi di test dai requisiti
- controlla e osserva il programma solo attraverso la sua interfaccia esterna (input/output)

In genere misura quanti input/output sono stati utilizzati

- non tutti gli input vengono utilizzati
- cerca di scoprire il maggior numero di difetti e di escludere quelli più critici

BlackBox

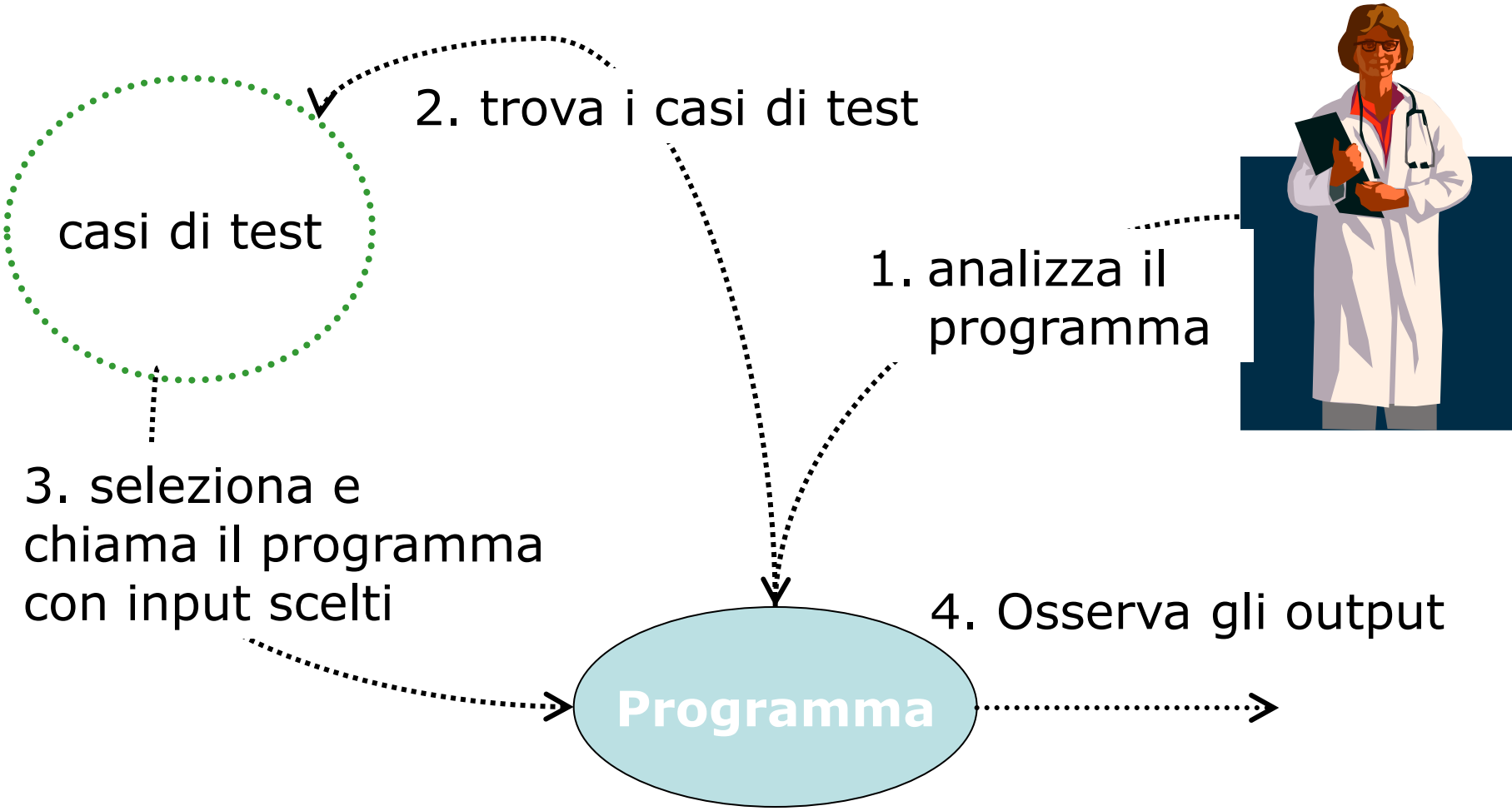


Program-Based Testing (1)

Passi principali

1. esamina la struttura del programma
   - quali i punti critici, le decisioni importanti,...
2. trova i casi di test (cioè input) che soddisfano un certo criterio di copertura
3. applica gli input (uno alla volta) e osserva il programma (output)
4. controlla che non si verifichino errori e che gli output siano quelli attesi

Program-Based Testing (2)



Valutazione del program-based testing

Vantaggi:

- il codice è una importante fonte di informazione disponibile e usabile

Limitazioni:

- non riesce a trovare errori di omissione
  - cosa succede se un programma dimentica di gestire un caso particolare? Guardando solo la struttura tale caso non verrà mai selezionato come caso di test
- non fornisce "test oracles"
  - come faccio sapere se l'output ottenuto è quello atteso?

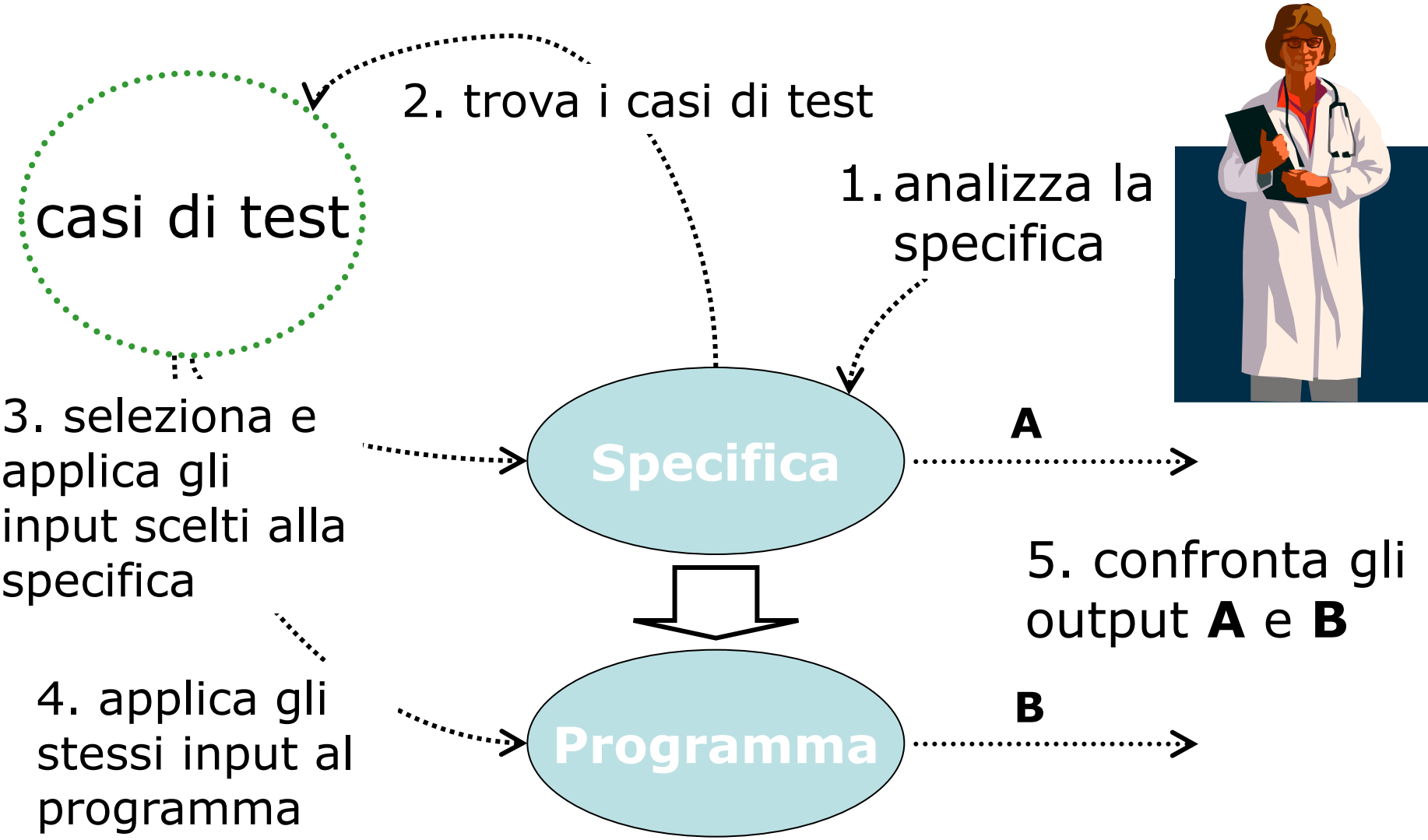
Test oracle: modo per stabilire se il test ha evidenziato un malfunzionamento oppure no

Specification-Based Testing (1)

Passi principali:

1. esamina la specifica dei requisiti del programma
2. seleziona un insieme di casi di test che soddisfano qualche criterio
3. applica questi input alla specifica e colleziona gli output **A** (attesi)
4. applica gli stessi input al programma e colleziona gli output **B** (osservati)
5. confronta gli output **A** con gli output **B** e controlla che siano uguali

Specification-Based Testing (2)



Valutazione del spec.-based testing

Principali vantaggi:

- la specifica funziona da **oracolo**

Principali limiti:

- le specifiche potrebbero non essere disponibili
  - potrebbe esserci solo il codice
- le specifiche devono essere "formali"
  - per generare i casi di test
  - per eseguire i casi di test selezionati per ottenere gli output
- maggiore sforzo rispetto al program-based test
- specifiche eseguibili

FONDAMENTI TEORICI DEL TESTING

Definizioni di base del testing

Un programma P è una funzione da un dominio D ad un codominio R:

$$P: \ D \rightarrow R$$

può essere parziale – non definita per qualche d in D

introduciamo un predicato OK:

OK(P,d) con d in D

- **OK**(P,d) = true se P è corretto per l'input d, cioè se produce P(d) corretto
- **OK**(P,d) = false altrimenti

OK(P) se P è corretto, se per ogni d in D **OK**(P,d)

Definizioni (2)

Rivediamo alcune definizioni e leghiamole a OK(P,d)

P sia il programma

FAILURE o malfunzionamento:

- Eseguo P con d e non ottengo P(d): o ottengo un altro valore P'(d) oppure il programma si blocca, …
- cioè not OK(P,d)

FAULT o difetto o bug:

- P è stato implementato in P'
- Se P' $\neq$ P, la diversità è il difetto

ERRORE:

- Il motivo del difetto

Definizioni (3)

- **Un caso di test (test case) o test è un elemento di D**
  - esempio:
    - se D è il dominio degli interi, un test è un intero,
    - se D è sequenze di interi, un test sarà un sequenza (possibilmente finita) di interi, ecc ...

- **Un test set T o test suite è un sotto insieme (finito) di D**
  - esempio:
    - D è un insieme di coppie di interi (D = int x int), T sarà un sottoinsieme di D, cioè un insieme di coppie di interi:
      $T \subseteq int \times int$

Esempio di test e test suite

Dato il metodo – programma P

```
static void foo(String s, int x)
```

un caso di test è una coppia String e int

- esempio `["pippo",3]`

un test set è un insieme di coppie String e int

- esempi:
  - `{["pippo",3]}` : insieme con un solo caso di test
  - con tre casi di test:
    `{["pippo",3],["",54657],["ccc",-10]}`

Test positivo e test negativo

Applico il test set T al programma P:

a) se **OK(P,T)**, ossia per ogni t in T vale **OK(P,t),**
il test set T non ha scoperto errori (**test negativo**)

b) se esiste t in T tale che **not OK(P,t)** (**test positivo**)

Test ideale e correttezza dei programmi

Inizialmente la ricerca cercò di trovare dei test che potessero dimostrare la correttezza dei programmi:

- testando il programma con un insieme finito T e non osservando difetti, si potesse essere sicuri che non ci fossero difetti
- tale test set (se esiste) è detto ideale

T è ideale se OK(P,T) → OK(P)

In questo caso l'esecuzione di T senza errori è prova di correttezza di P

Come faccio a trovare un T ideale?

Testing esaustivo

Un test ideale è quello esaustivo con T = D

OK(P,T) → OK(P,D)

è fattibile? Considera questo esempio:

```
static int sum(int a, int b) return a + b;
```

- un int è un numero binario di 32 bit, quindi ci sono solo

  $2^{32} \times 2^{32} = 2^{64}$ ca 10^{21} test
  ad un nanosecondo (10^{-9}) per test case
  circa 30000 anni.

- se D è infinito? (ad esempio un sistema reattivo)

Il test esaustivo <u>non è fattibile</u>: $T \subset D$

Devo selezionare un sottoinsieme di D. Come fare?

Osserva

- Un test esaustivo non è praticabile
- È necessario stabilire strategie di testing e criteri di selezione di casi di test significativi
  - Un test che non rileva malfunzionamenti manifesta non la correttezza del programma, ma la sua inadeguatezza
- Cerchiamo criteri adeguati, ossia criteri che ci permettono di selezionare test set (minimali) adeguati (o *significativi*) tale da essere in grado di evidenziare malfunzionanti
  - Il concetto di *adeguatezza di un test* non può essere formalizzata in modo assoluto, in quanto è un'approssimazione empirica del concetto di test ideale

Test Criteria

Test criteria o criterii di adeguatezza sono funzioni del tipo:

$$C: P \times T \rightarrow \{True, False\}$$

con P programma e T test set

- *C(P,T) è True* se e solo se T è **adeguato** a trovare ogni difetto in P secondo il criterio C

Un criterio può essere anche usato come generatore di test set adeguati:

$C: P \rightarrow T$ tale che $C(P,T) = True$

dato P genera un test set T adeguato secondo C

Esempio di Test Criteria

Sia P un programma con dominio D (interi):

- un test set *T sarà un insieme di interi*
- un criterio C dovrà dire quando T è adeguato a testare P, ossia:
- *C sarà un predicato sugli interi che permetterà di costruire un T adeguato a testare P secondo C*

Es: sia C un criterio che stabilisce T adeguato se T contiene un numero positivo e uno negativo:

- C se $\exists$ t $\in$T t>0 and $\exists$ t $\in$T t<0
  - test set adeguati: {-1,1}, {-1,-5, -10, -20, 5}
  - test set non adeguati {-1,-5}, {0,15}

Esempio di Criterio di testing

Dato P (frammento):

```
f(x,y,z) = …
  if (x < 0 && y > 0) then
          if (z > x && z < y) then S1 else S2
  else S3
```

Stabilire un criterio adeguato per eseguire lo statement S1:

```
C = {(x,y,z)| x < 0 & y > 0 & z > x && z < y)}
```

Ed adeguato per eseguire S3?

Criterio affidabile (1)

Un criterio C è affidabile per P se per ogni coppia di test set T1 e T2 adeguati secondo il criterio C, se T1 individua un malfunzionamento, allora anche T2 e viceversa

- OK(P,T) non dipende dal T particolare selezionato
- tutti i test set o riescono a trovare i difetti, oppure non riescono a trovarne
- posso prendere il più piccolo T: se scopro errori con uno più grande, li scoprirei anche con uno più piccolo

Criterio affidabile (2)

- Un criterio affidabile produce sempre risultati consistenti e ugualmente utili
- Non è detto però che un criterio affidabile riesca a scoprire malfunzionamenti:
  - considera raddoppia:

```
int raddoppia(int x) { return x*x ;}
```

  - un criterio C1 che seleziona solo sottoinsiemi di {0,2} è affidabile
  - T1 ={0,2} , T2 ={2} …

- Affidabilità è importante ma non è sufficiente

Criterio valido

Un criterio C è valido per P se, qualora il programma P non sia corretto, esiste almeno un test set T che soddisfa C che è in grado di individuare il difetto

- formalmente
  $\exists$ T tale che C(P,T) and not OK(P,T)

- un criterio valido riesce (ma non garantisce) a costruire casi di test che possono trovare difetti

Esempio di criterio valido

considera raddoppia:

```
int raddoppia(int x) { return x*x ;}
```

- un criterio C2 che seleziona

  qualsiasi $T \subseteq \{0,1,2,3,4\}$

  C2 è valido, basta prendere {1} o {0,1} o {1,3}

  ma non affidabile: {0,2} non scopre il difetto

- C3: $\exists t \in T$ tale che t >= 3

  esempi di T: {3}, {0,2,3}, {-4,3,6,10} ,…

  C3 è valido : scopro il difetto

  e affidabile : scopro sempre il difetto

Criterio ideale

Def: Un criterio C valido e affidabile è detto criterio ideale

Esempio:

programma raddoppia:

```
int raddoppia( int x ) {
        return x*x ;
}
```

un criterio che selezioni casi di test con almeno un numero in T che sia >= 3 è valido e affidabile, quindi ideale

Teorema di Goodenough e Gerhart

Dato un criterio C, un programma P

Se C è *affidabile* per P e
C è *valido* per P e } C è *ideale*
T test suite selezionato con C e
T non trova malfunzionamenti in P: *OK(P,T)*

=> **OK(P), T è ideale**

Se un programma P passa un test set T che è selezionato con un criterio affidabile e valido (si dice in questo caso IDEALE), allora P è corretto

Esempio di criterio valido e affidabile

```
int raddoppia( int x ) {
          return x*x ;
  }
```

Un criterio che selezioni casi di test con un numero >= 3 è valido e affidabile.

Però:

- di questo programma sappiamo già il difetto
- sappiamo già esattamente cosa deve fare

Possiamo trovare criteri in generale validi e affidabili?

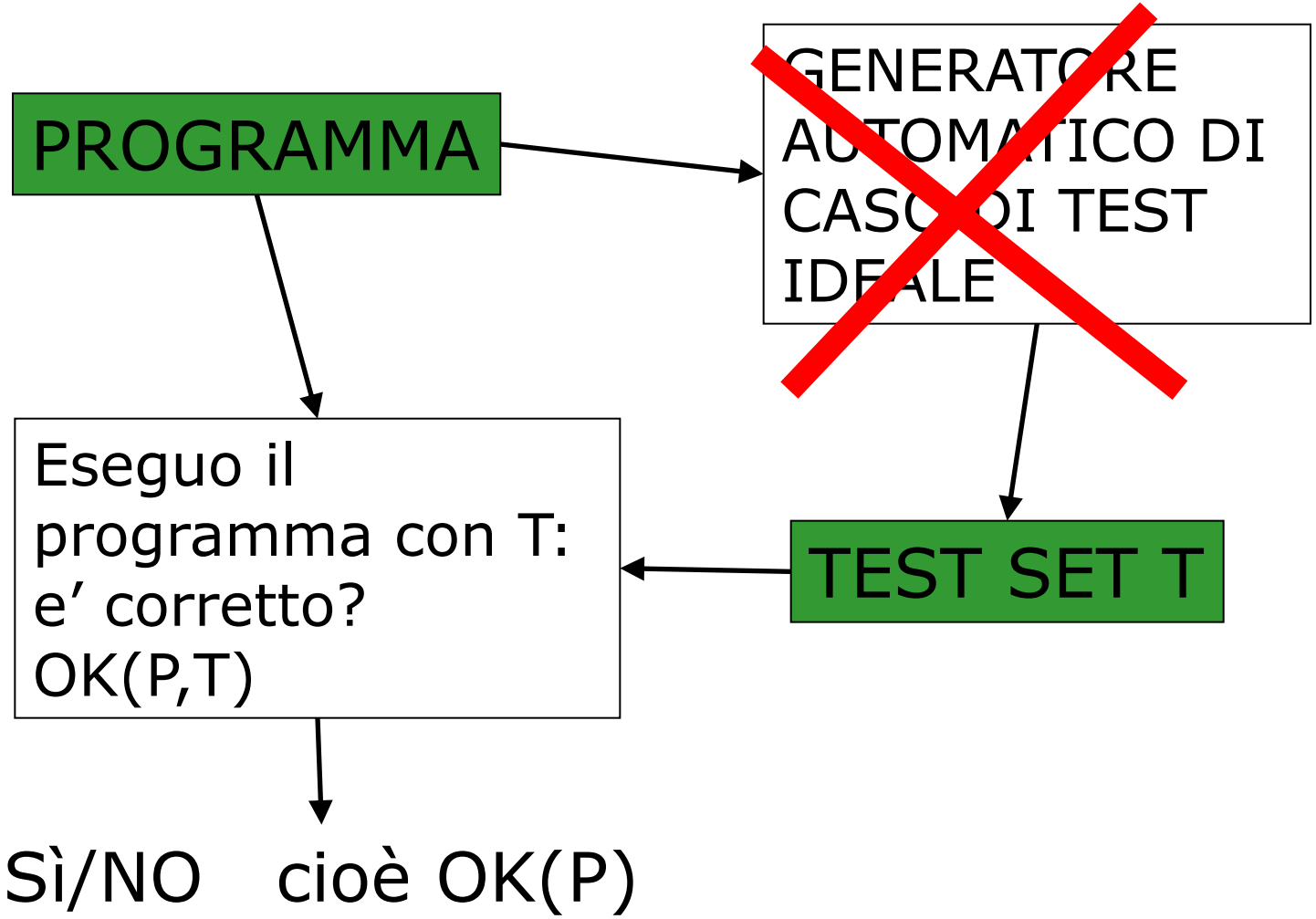
- Un criterio che richiede T = D è ideale ma impraticabile
  - (come visto in precedenza)

Howden

NON POSSO IN MODO ALGORITMICO TROVARE CASI DI TEST IDEALI:

Teorema di Howden
non esiste un algoritmo che dato un programma P generi un test ideale finito.

Correttezza mediante testing: possibile?



IL TESTING NON PUÒ DIMOSTRARE CORRETTEZZA DEL SW

Dijkstra:
Il test di un programma può rilevare la presenza di malfunzionamenti ma mai dimostrarne l'assenza.

ESERCIZI

(5.6-5.10)

Esercizi

Dato il seguente programma che dovrebbe fare la somma di due numeri, ma che contiene un difetto:

```
int somma (int x, int y) {
   if (x>0 && y>x) return x*y;
     else return x+y;
}
```

determinare un criterio per la costruzione di test set ideali.

Dare la definizione di criterio valido, affidabile ed ideale. Determinare un criterio ideale per il seguente programma che dovrebbe restituire il valore della funzione `f(x,y)=x*(y*10)` per valori x,y interi, crescenti e maggiori di 1, 0 altrimenti, ma che contiene un errore:

```
int foo (int x, int y) {
   if (x>1 & y>x) return x*(y+10);
     else return 0;
}
```

Criteri di Test non ideali

Se non posso definire criteri ideali per il test, cosa posso definire in alternativa?

Definizione di "empirical" test criteria

- criteri che non sono ne' validi ne' affidabili ma si sono dimostrati utili

Test criteria come "stopping rule":

- ho testato abbastanza (posso essere *confidente* – ma non certo - che P sia corretto?)

Ad un test set T associo una misura di copertura: P x T -> [0, 1] (Test adequacy)

Testing strutturale dei programmi

Nel testing basato sulla struttura dei programmi:

- i criteri di test sono definiti considerando il solo codice sorgente
- il sorgente viene usato per:
  - generare i casi di test
  - decidere quando si ha testato abbastanza

Copertura e flusso di controllo

- I criteri di test strutturali sono definiti considerando la "copertura" del codice

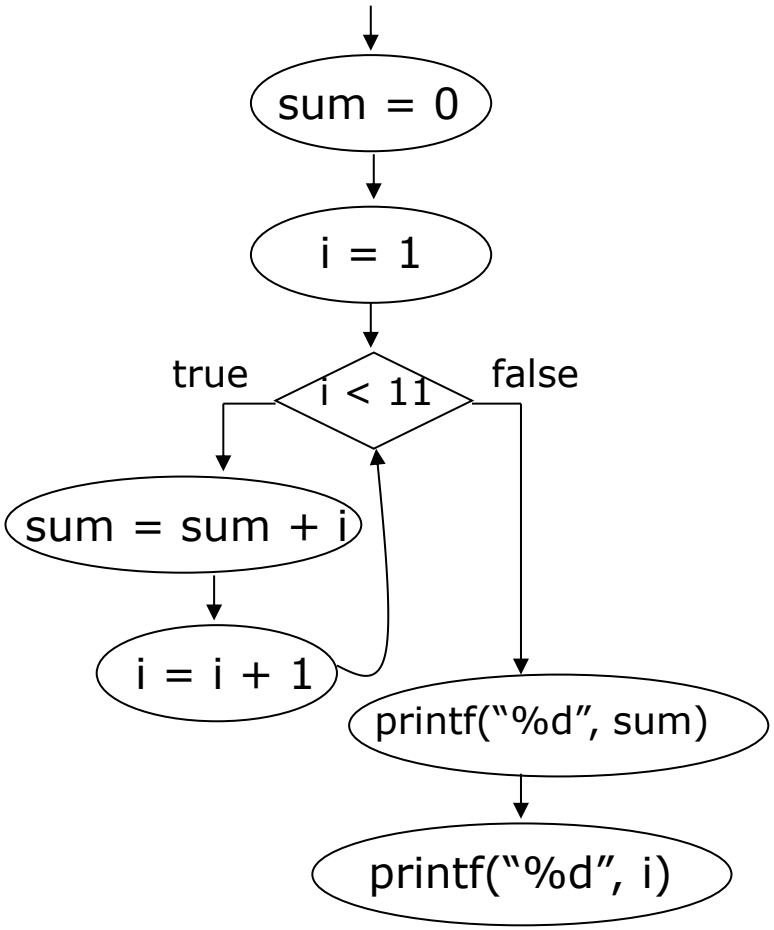
Per **copertura** di un programma si intende la parte del programma che viene eseguita dai casi di test

- In particolare intendiamo la struttura del codice come flusso di controllo del programma
  - la copertura sarà relativa al flusso di controllo

Flusso di controllo

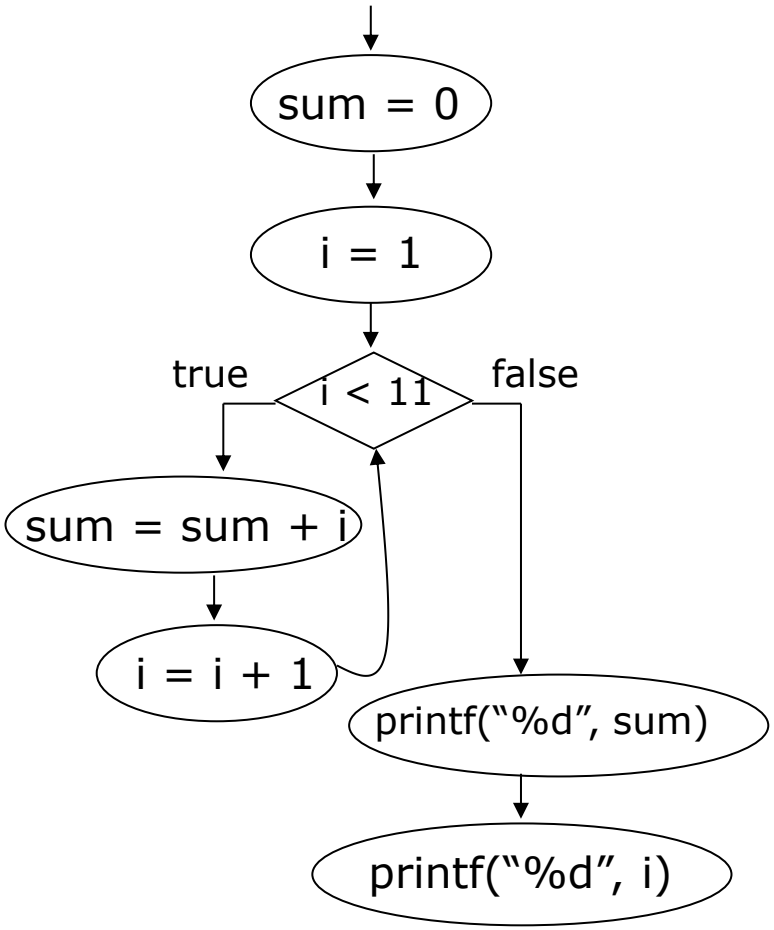
Il comportamento di un programma può essere rappresentato tramite il suo grafo del flusso di controllo

- rappresentazione grafica del codice sorgente,
- rappresenta qualsiasi esecuzione possibile
  - ignora i particolari valori di variabili che possono variare da una esecuzione a quella successiva



Flusso di controllo

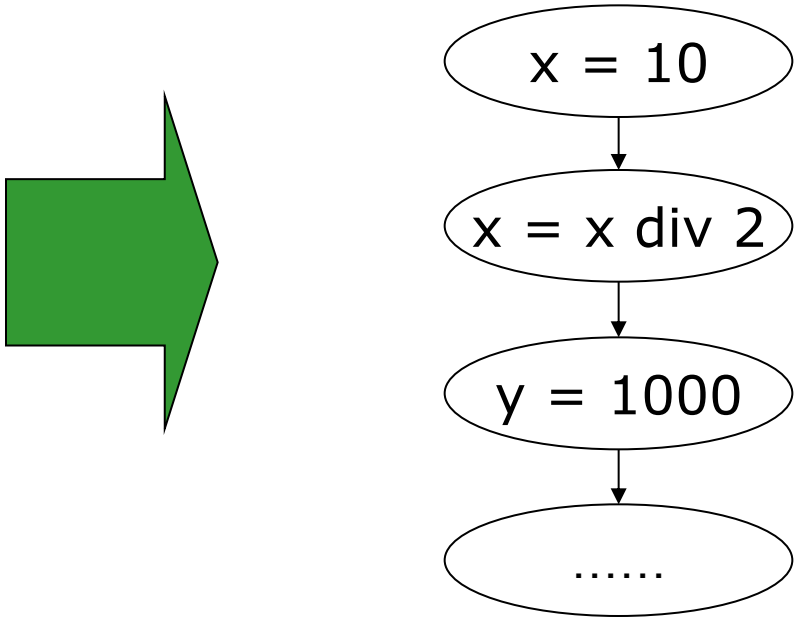
- ogni istruzione è un nodo del grafo
- ogni istruzione è collegata alla sua successiva mediante una freccia



Flusso di controllo: esempio (1)

Nel caso di una semplice sequenza di istruzioni si ha un grafo semplice e intuitivo come quello dell'esempio:

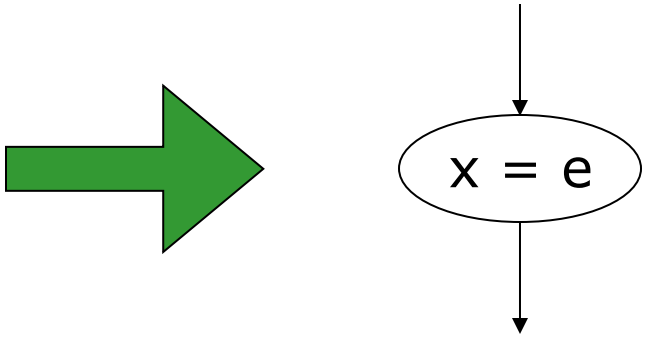
```
x e y sono interi

x = 10;
x = x div 2;
y = 1000;
……
```



Istruzioni semplici

Le istruzioni di assegnamento, di lettura, scrittura e di return li rappresentiamo con nodi circolari

Istruzione di assegnamento
`x = e;`



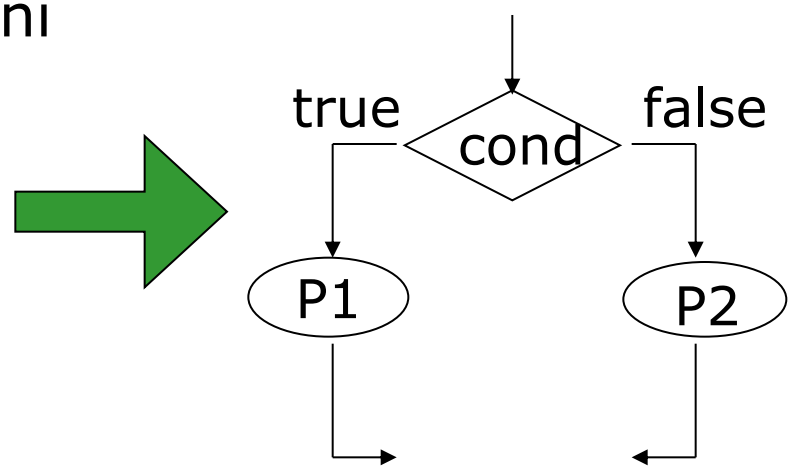
Anche chiamate di metodi esterni verranno rappresentate con semplici nodi circolari

Nodi Decisione – Istruzione If

Le istruzioni condizionali vengono rappresentate:

- da un rombo che contiene la decisione
- due cerchi con le due istruzioni (il ramo else è opzionale)
- due frecce marcate true e false che vanno dalla decisione alle due istruzioni

```
if (cond) then P1 else P2
```



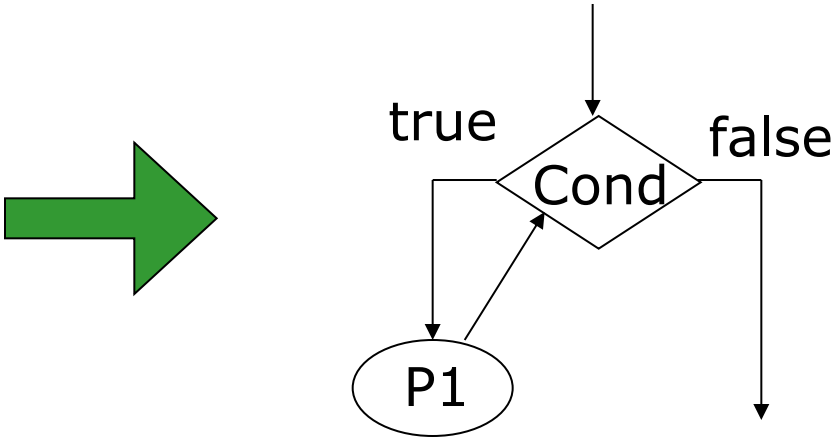
rappresenteremo le decisioni sempre con dei rombi

Ciclo While

I cicli while vengono rappresentati da:

- una decisione che è la condizione del while
  - con una freccia “false” che va all’istruzione successiva
  - con una freccia “true” che va alle:
- istruzioni del corpo del while
  - da eseguire se la decisione è vera
- finite le istruzioni del corpo si ritorna alla valutazione della decisione

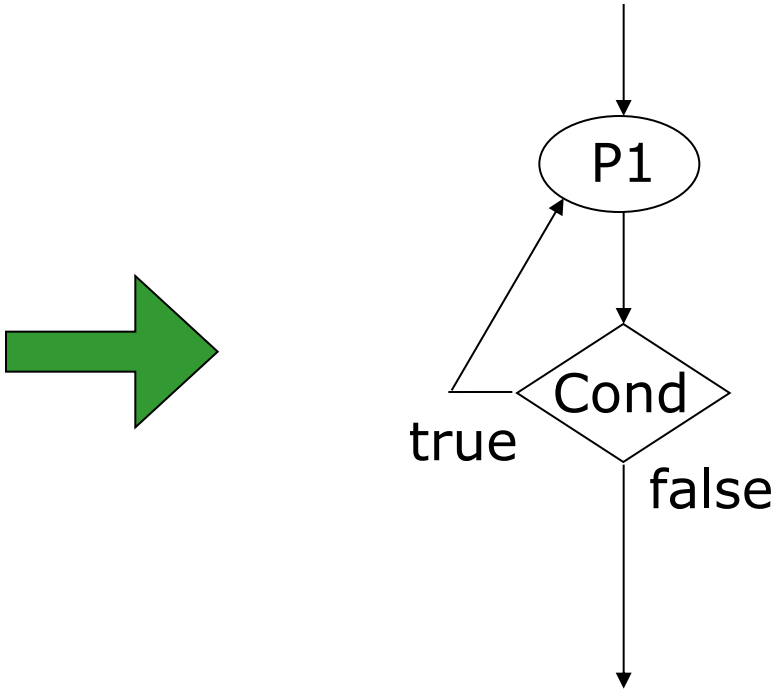
```
while (cond) P1
```



Ciclo Do-While

I cicli do-while vengono rappresentati come il while ma la decisione è posta alla fine

```
do P1
while (cond)
```

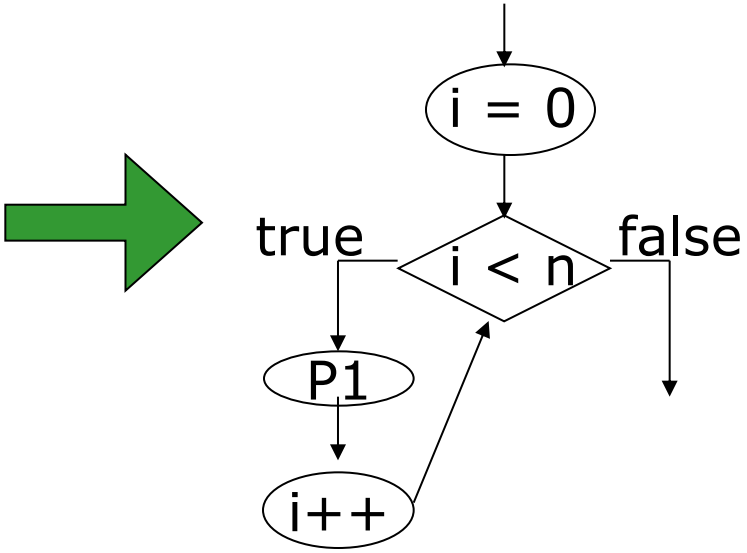


Cicli For

I cicli for sono rappresentati con il loro equivalente ciclo while

- l'istruzione di inizializzazione viene messa per prima
- la decisione è la condizione del ciclo for
- l'incremento viene eseguito come ultima istruzione del corpo
- dopo l'incremento si torna a valutare la decisione

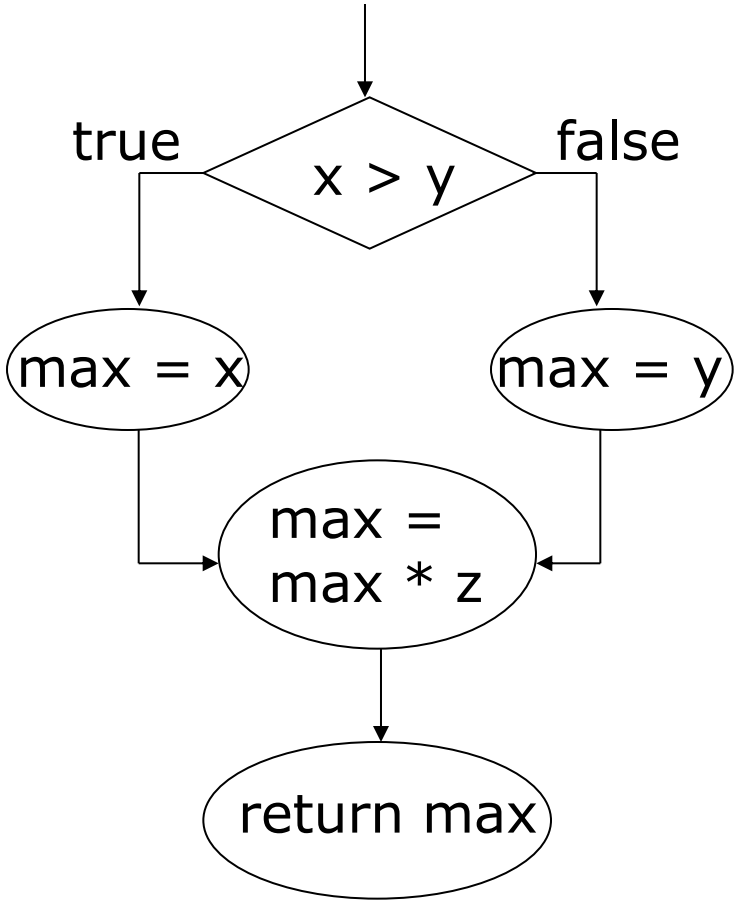
```
for (i = 0; i++; i < n) P1
```



Flusso di Controllo: esempio (2)

Esempio di metodo main e suo grafo del flusso di controllo

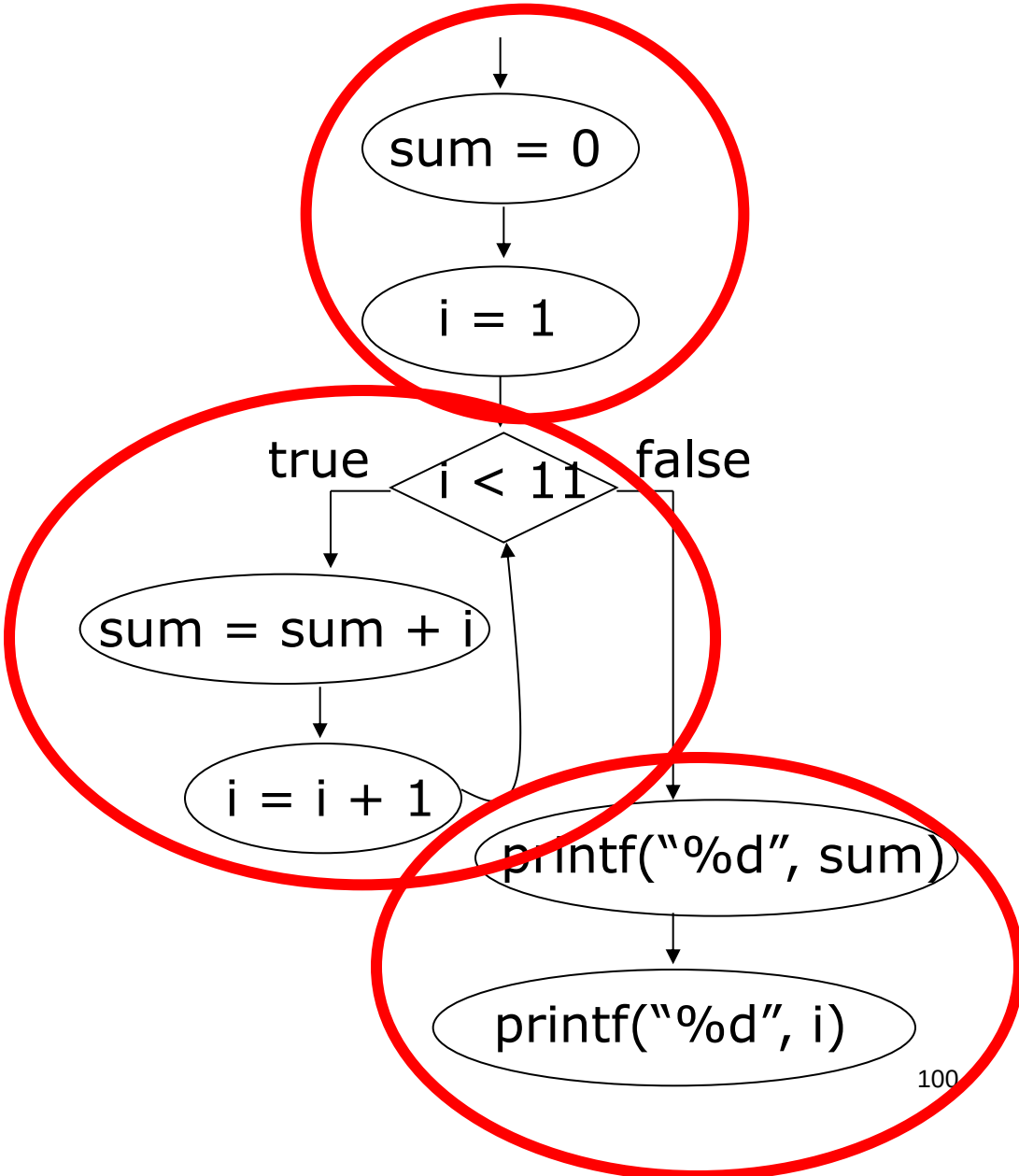
```
int main(int x, y, z) {
    int max;

    if (x > y)
        max = x;
    else
        max = y;
    max = max * z;
    return max;
}
```



Flusso di controllo: esempio (3)

Esempio con while

```
int main() {
  int sum, i;

  sum = 0;
  i = 1;
  while (i < 11) {
        sum = sum + i;
        i = i + 1;
  }
  printf("%d",sum);
  printf("%d",i);
}
```



Il grafo di flusso di controllo come astrazione

Il flusso di controllo può essere considerato un'astrazione delle esecuzioni del programma

- ignora il particolare cammino che verrà preso per certi valori delle variabili
- rappresenta tutte le possibili esecuzioni
  - ad ogni esecuzione verrà eseguita un sequenza di nodi che possiamo disegnare con una linea nel grafo
  - per ogni computazione di un programma c'è la corrispondente computazione nel grafo del flusso di controllo

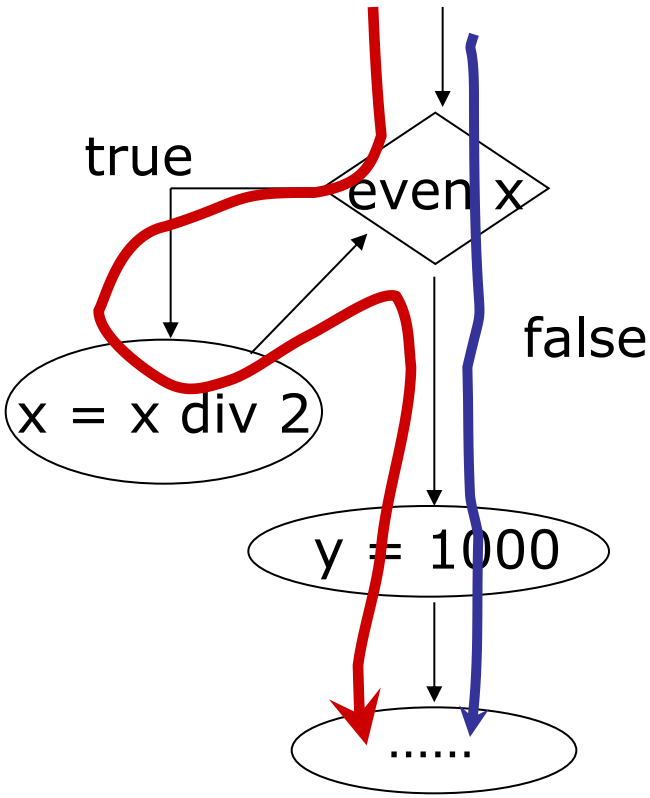
Esempio di esecuzione e sua rappresentazione nel grafo

etichette

```
        int x, y;
P1:         while (even x)
P2:                 = x div 2;
P3:         y = 1000;
P4:         ......
```

Esecuzione con x = 3, y = 1

P1 <3,1>
↓
P3 <3,1>
↓
P4 <3,1000>

Esecuzione con x = 6, y = 1

P1 <6,1>
↓
P2 <6,1>
↓
P1 <3,1>
↓
P3 <3,1>
↓
P4 <3,1000>